UART(RX,TX) + FIFO Loopback



이름: 조수영

날짜: 2026.10.26

Contents

01 개요

02 블럭 다이어그램

03 Related Technical

04 시뮬레이션 시나리오

05 시뮬레이션 결과

06 동작 영상

07 결론 및 고찰

프로젝트 개요

설계 목적

UART RX · TX와 FIFO를 설계한 뒤, 루프백 구조로 묶어 전체 데이터 경로를 한 번에 검증



하드웨어 · 소프트웨어

- 보드 : Basys 3 (AMD Xilinx Artix-7), 100 MHz 클럭
- HDL · 개발 환경 : Verilog, Vivado
- UART 테스트 프로그램 : ComportMaster

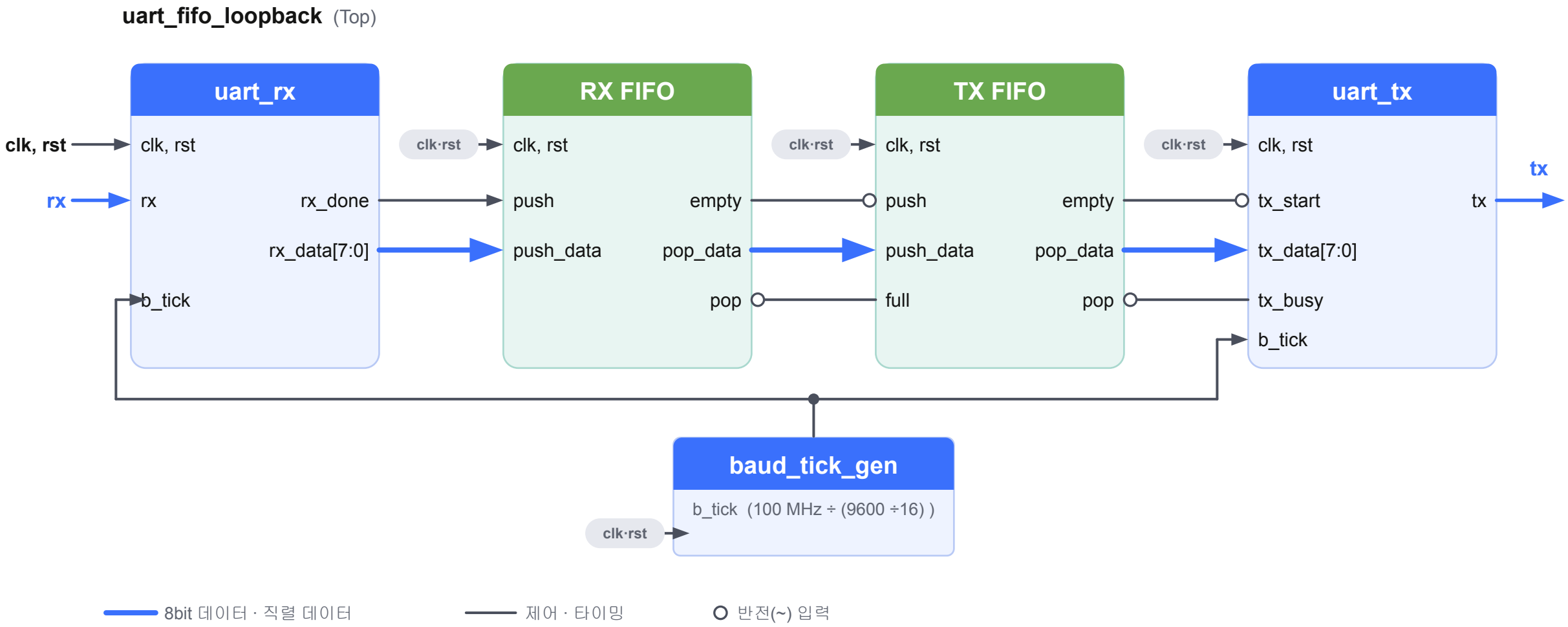
설계 사양

시스템 클럭	100 MHz
통신 형식	9600 bps, 8N1 (Data 8 · Parity 없음 · Stop 1)
오버샘플링	16배, 651분주 (오차 +0.006%)
FIFO	깊이 4 × 8bit, RX · TX 각 1개 (최대 8바이트)
리셋	비동기, 액티브 하이

결과 요약

UART +FIFO 의 루프백 구조로 설계, 랜덤한 값을 256번 넣은 검증테스트 pass
rx 감지지연 랜덤 x시 오차 -4.8~6.9%에서 정상 수신 | 감지 지연 랜덤 시 오차 -4.8~6.2%에서 정상 수신

블럭 다이어그램


uart_fifo_loopback (Top)
uart_rx
clk, rst
rx
rx
rx_done
rx_data[7:0]
b_tick
clk·rst
RX FIFO
clk, rst
push
empty
push_data
pop_data
pop
clk·rst
TX FIFO
clk, rst
push
empty
push_data
pop_data
full
pop
clk·rst
uart_tx
clk, rst
tx_start
tx
tx_data[7:0]
tx_busy
b_tick
tx
baud_tick_gen
b_tick (100 MHz ÷ (9600 ÷16))
clk·rst
8bit 데이터 · 직렬 데이터
제어 · 타이밍
반전(~) 입력

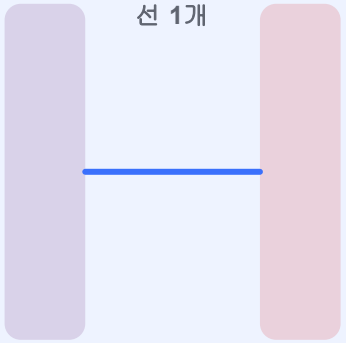
Related Technical

UART

Universal Asynchronous Receiver / Transmitter — 병렬 ↔ 직렬 변환으로 데이터를 주고받는 비동기 직렬 통신 인터페이스

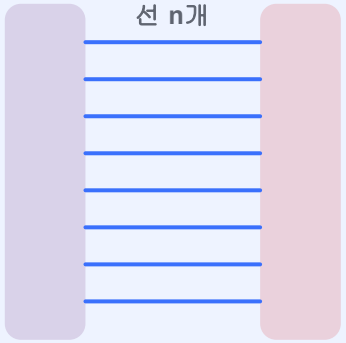


직렬 통신 (Serial)



- 한 선으로 1bit씩 차례로 전송
- 배선·핀 수가 적어 구조 단순, 비용 낮음
- 선 간 도착 시간 차이가 없어 장거리에 유리
- 같은 클럭에서는 상대적으로 느림

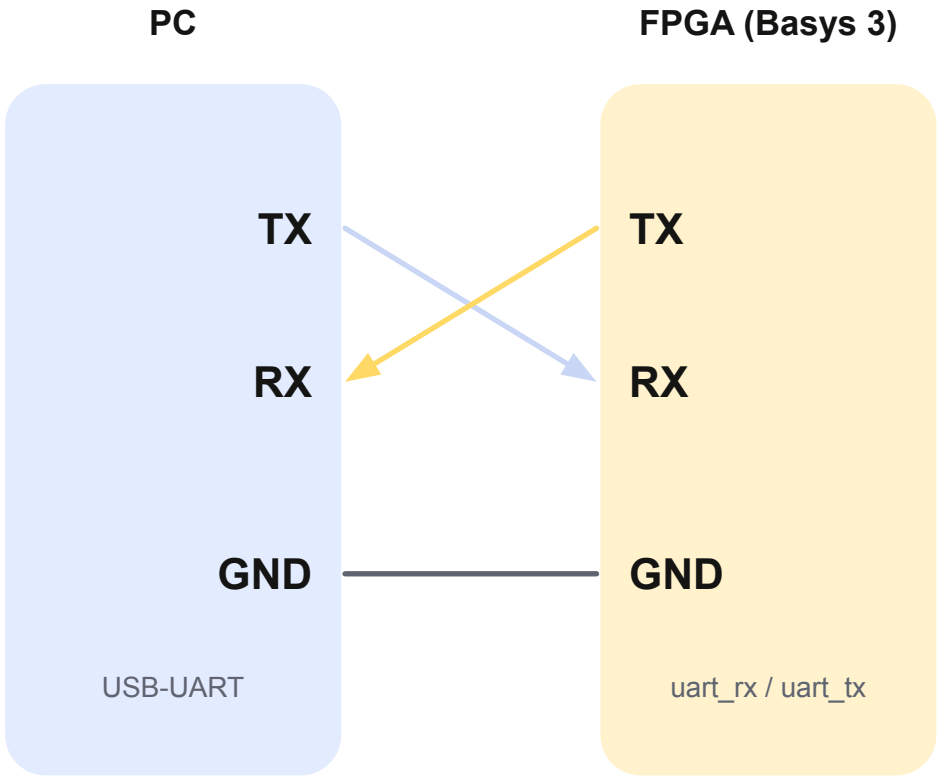
병렬 통신 (Parallel)



- n개의 선으로 nbit 동시 전송
- 같은 클럭에서 n배 빠름 (단거리)
- 거리 증가 시 선마다 도착 시간 차이 (skew)
- 선 간 간섭(crosstalk)·노이즈, 배선 비용 증가

Related Technical

RX · TX 연결과 특징



TX는 상대 RX로 교차 연결, GND는 공통 연결

PC → FPGA (파랑) · FPGA → PC (주황)

1. **송신 TX · 수신 RX 분리 → Full-duplex**
   송신선과 수신선이 따로 있어, 보내는 **동시**에 받을 수 있음

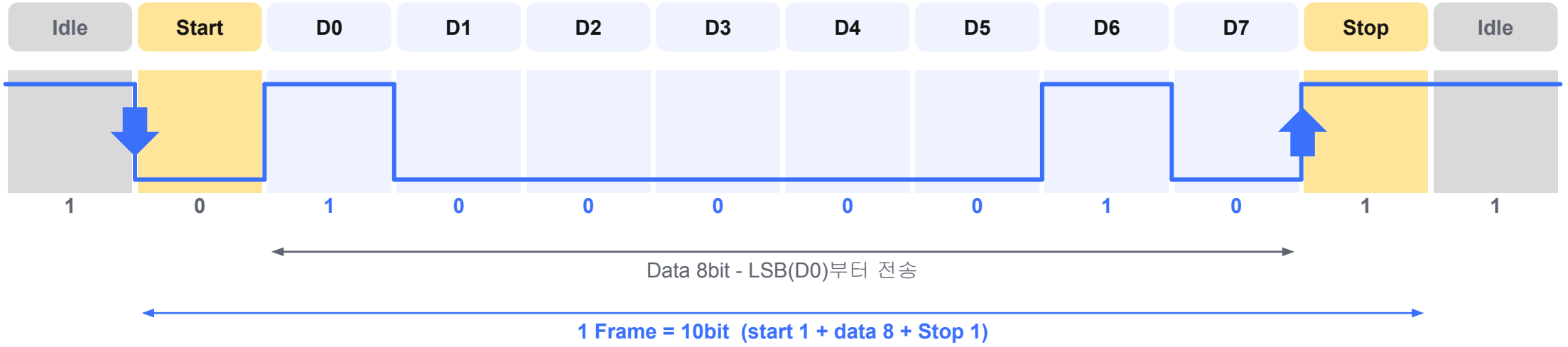
2. **비동기 방식**
   보레이트(baud time)만 미리 약속하고 **각자 자기 타이머**로 비트 시간을 잼. 매 프레임 스타트 비트로 타이밍을 다시 맞춤

3. **GND 공통 연결**
   전압은 기준점과의 차이 → 두 장치의 0V 기준을 맞추고 전류가 돌아갈 귀환 경로를 제공

4. **TX = PISO, RX = SIPO**
   송신: 병렬 8bit를 한 번에 받아 1bit씩 출력 / 수신: 1bit씩 모아 8bit 병렬로 출력

Related Technical

UART 데이터 프레임



1 프레임 구성

Idle HIGH(1) — 대기 상태

Start 1→0

Data 5~9bit, 보통 8bit

Parity <선택> 1의 개수로 오류 검출

Stop HIGH

2 LSB First 전송

[7:0]data = **0100 0001**

전송 순서 : **1 0 0 0 0 0 1 0**

- 하위 비트(D0)부터 차례로 송신
- 수신 측은 오른쪽 시프트하며 원래 순서로 복원

3 Baud Rate · Baud Time

- Baud rate : 초당 비트 수 (9600 bps)

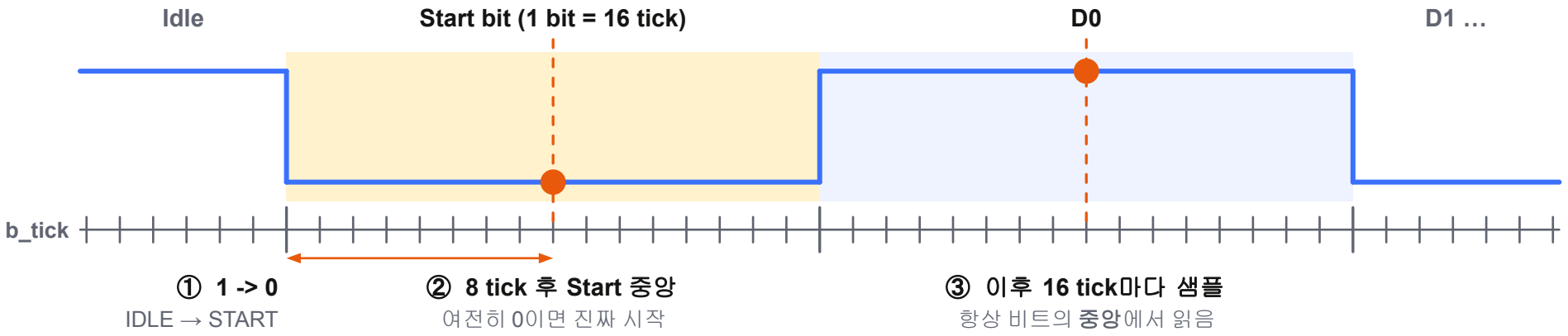
Baud time = 1 / 9600 ≈ **104.16 µs**

1 Frame ≈ 1.04 ms → 최대 **960 Byte/s**

- 송·수신 설정이 다르면 데이터 깨짐

Related Technical

Baud Rate 생성과 16배 오버샘플링



1 왜 오버샘플링인가

- 클럭 선이 없어 수신 측은 비트 경계를 직접 알 수 없음
- 1bit를 여러 구간으로 쪼개 하강 엣지 위치를 정밀하게 찾음
- 경계가 아닌 비트 중앙에서 읽어 클럭 오차·노이즈에 여유 확보

2 Baud Tick 생성 (100 MHz)

F_COUNT = 100,000,000 / (9600 × 16)

- Baud tick 16번 마다 1클럭 폭 b_tick 펄스

3 8배 vs 16배

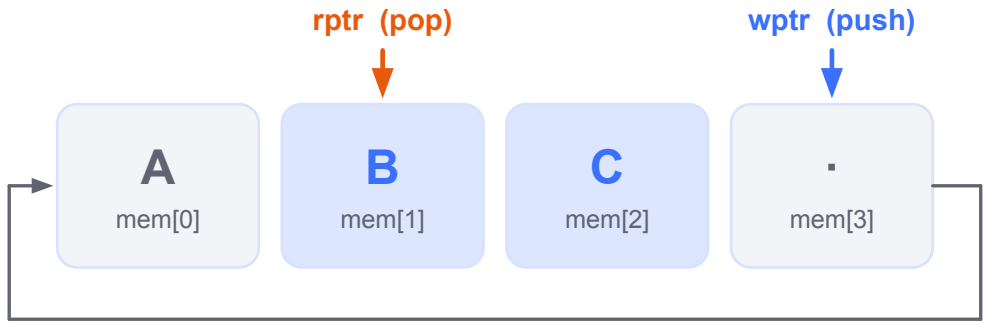
8x 1/8 bit 정밀도, 중앙까지 4 tick
카운터가 작고 고속 보레이트에 유리

16x 1/16 bit 정밀도, 중앙까지 8 tick
오차 여유가 커 가장 널리 쓰임 → 본 설계 적용

Related Technical

FIFO (First In, First Out)

먼저 들어온 데이터가 먼저 나가는 버퍼 — 쓰는 쪽과 읽는 쪽의 속도/타이밍이 다를때 데이터를 보관



마지막 칸 다음은 0번 칸 (원형 버퍼) · 데이터 대신 포인터만 이동

이 설계의 FIFO

- register_file : 깊이 4 × 8bit 메모리
- control_unit : 2bit 포인터 wptr · rptr, full · empty 플래그
- 쓰기는 클럭 동기 (we = ~full & push)
- 읽기는 조합 출력 (pop_data = mem[rptr])

1 push (쓰기)

- wptr 칸에 push_data 저장 후 wptr + 1
- full이면 쓰기를 막아 덮어쓰기 없음

2 pop (읽기)

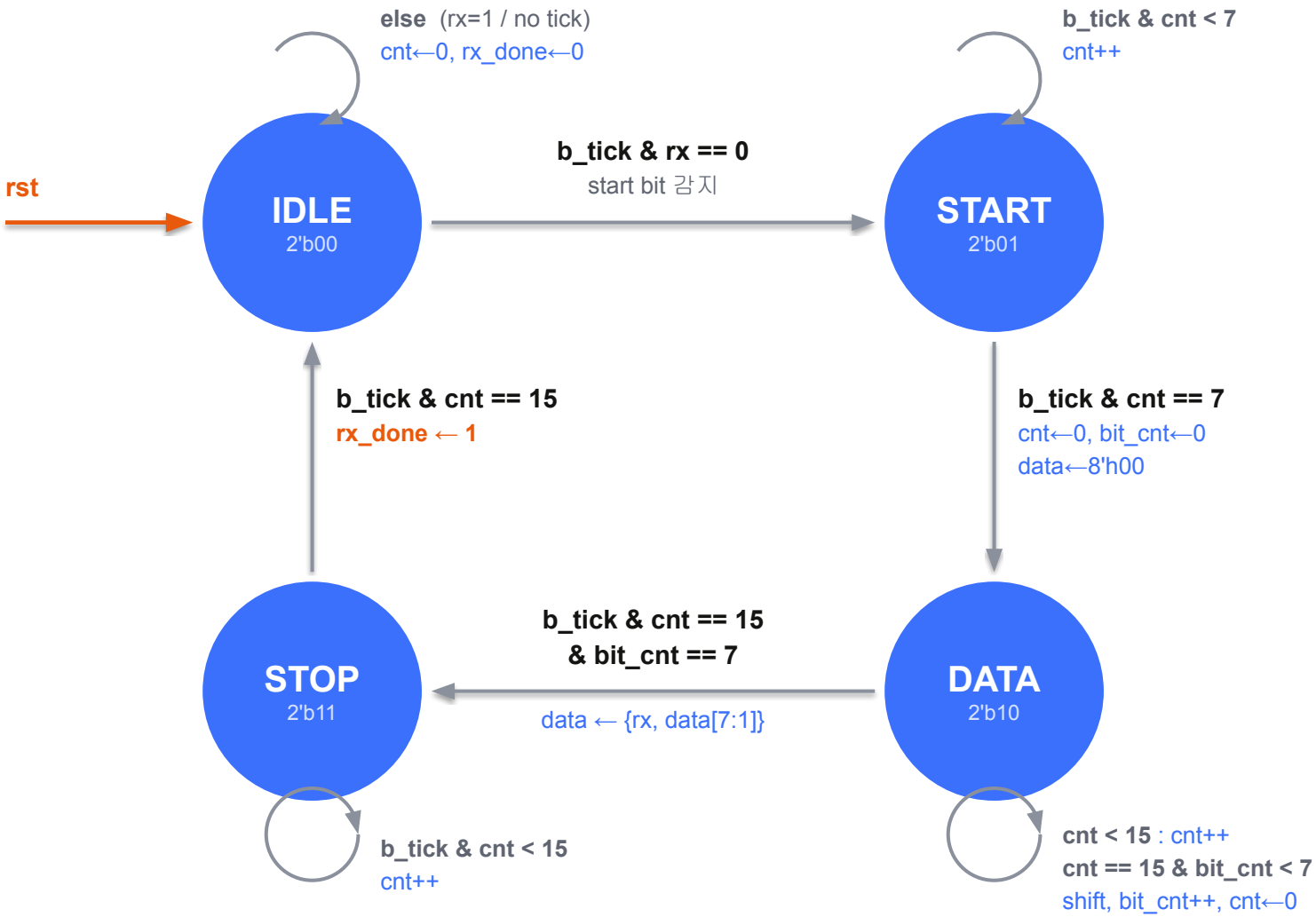
- pop_data는 항상 rptr 칸을 출력
- pop 시 rptr + 1, empty면 무시

3 full · empty 판정

- push 후 wptr + 1 == rptr → full
- pop 후 rptr + 1 == wptr → empty
- 포인터가 같을 때 두 상태를 플래그로 구분

Related Technical

RX FSM



상태별 동작 b_tick = 16× baud

IDLE 대기

b_tick 시점에 rx = 0 (start bit) 감지 시 START로 이동

START **start bit 중앙 정렬**

8 tick(½ bit) 카운트 → 비트 중앙에서 샘플링하도록 위상 맞춤

DATA **8bit 수신 (LSB first)**

16 tick마다 rx 샘플링 후 오른쪽 시프트, bit_cnt = 7이면 STOP

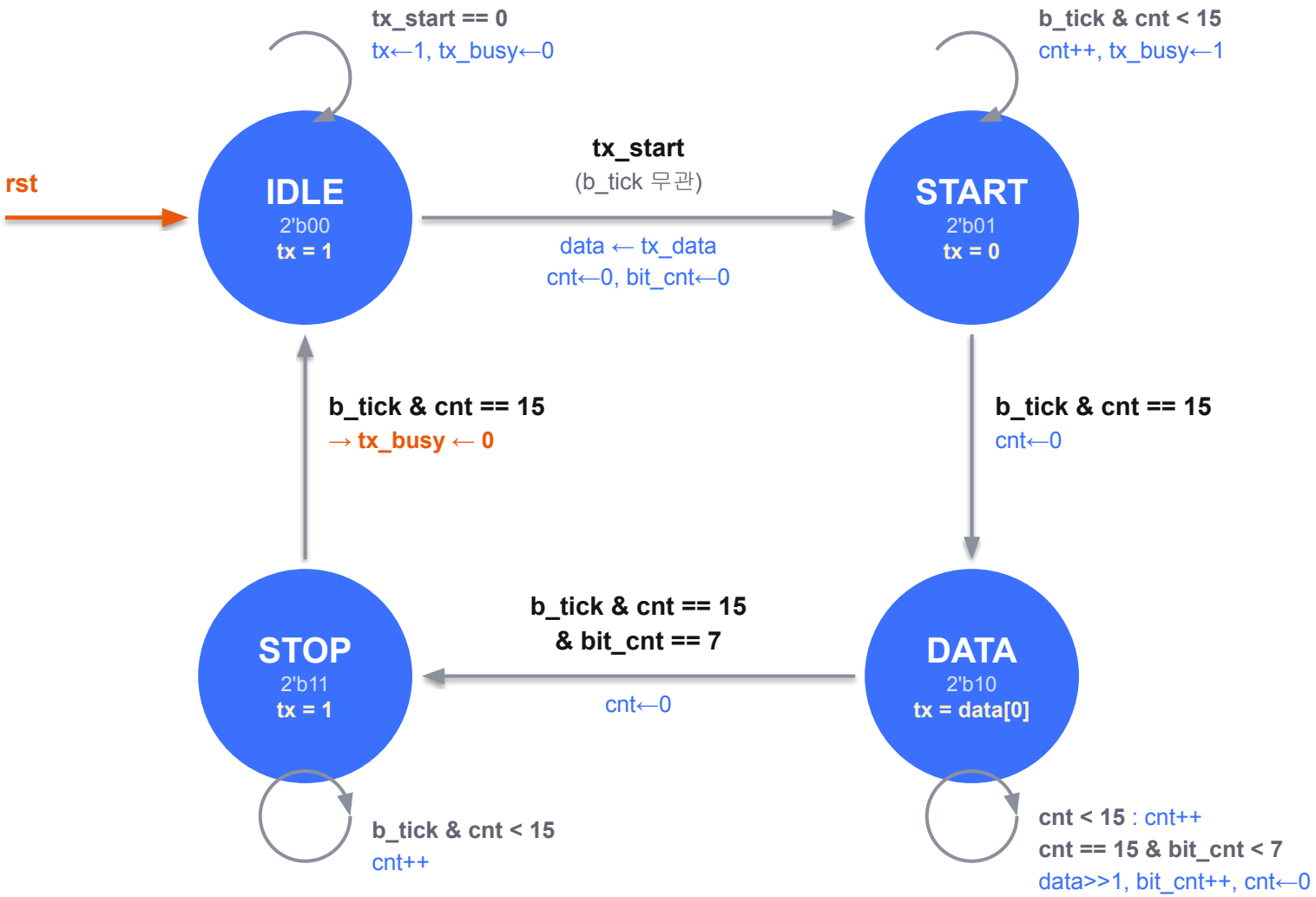
STOP **stop bit 대기**

16 tick 후 rx_done = 1 ,rx =high

※ 모든 전이는 b_tick일 때만 평가, 조건 불만족 시 상태 유지

Related Technical

TX FSM



상태별 동작 b_tick = 16× baud

IDLE 대기 (tx = 1)

tx_start가 들어오면 tx_data를 래치하고 START로 이동

START start bit 송신

tx = 0, tx_busy = 1 출력. 16 tick(1 bit) 유지 후 DATA로 이동

DATA 8bit 송신 (LSB first)

tx = data_reg[0] 출력, 16 tick마다 오른쪽 시프트. bit_cnt = 7이면 STOP

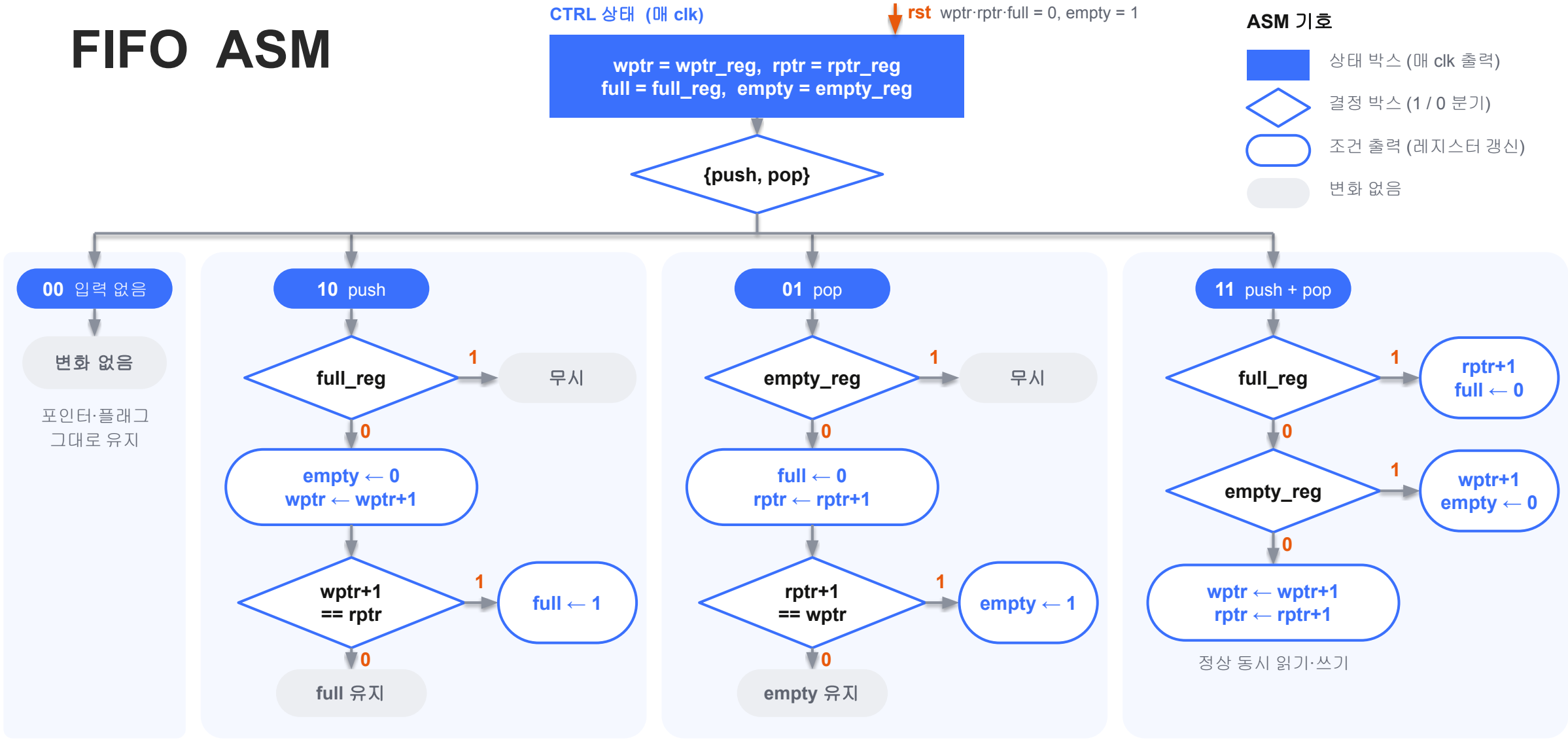
STOP stop bit 송신

tx = 1, 16 tick 유지 후 IDLE로 복귀 → tx_busy = 0

※ IDLE→START만 tx_start로 즉시 전이, 나머지는 b_tick일 때 평가

Related Technical

FIFO ASM



※ 모든 경로는 다음 clk에 CTRL 상태로 복귀

시뮬레이션 시나리오

시나리오 개요

구분	목적	주요 입력	확인 항목
S1	단일 바이트 루프백의 동작·타이밍 확인	expected_data = 8'b01010101	상태 전이, 16 tick 간격, 시프트, FIFO 전달, 데이터 일치
S2	다양한 데이터 값에서 루프백 정상 동작 확인	랜덤 8bit 값 256회	회차별 데이터 일치, pass / fail 집계
S3	보레이트 오차 허용 범위 검증	송신 bit_period = BAUD_BIT_PERIOD × (1 + skew)	- skew별 pass / fail 개수 - 100% pass가 유지되는 최대 오차, 첫 fail 오차 - 위상 고정 / 랜덤에 따른 한계 차이
공통 조건	clk 100 MHz (#5 토글) · 9600 bps 8N1 · 1 bit = 104,160 ns (651 × 16 tick × 10 ns)		

시뮬레이션 시나리오

S1 단일 바이트 검증

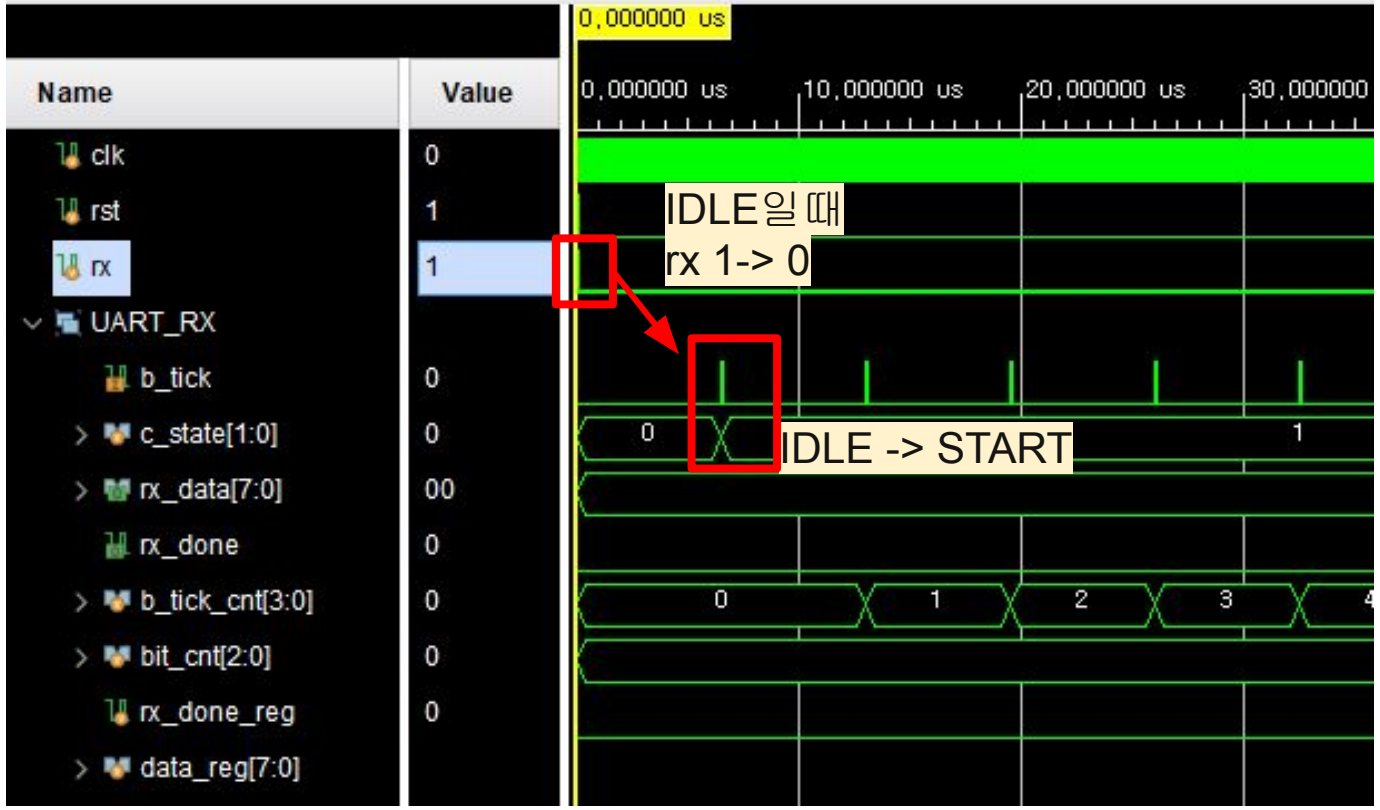
ID	검증 항목	입력 · 조건	기대 결과
1	IDLE → START	rx 1 → 0 (스타트 비트)	첫 b_tick에서 START 진입
2	START → DATA, D0 읽기	START에서 b_tick_cnt = 7	8 tick 후 DATA, D0 비트 중앙 부근에서 읽음
3	16 tick 간격 읽기 · 시프트	DATA에서 b_tick_cnt = 15	읽기 간격 104.16 µs (baud time), data_reg 우시프트 저장
4	DATA → STOP	b_tick_cnt = 15 & bit_cnt = 7	STOP 진입
5	수신 데이터	입력 0101_0101	rx_data = 0101_0101
6	FIFO 전달	rx_done 1클럭	RX FIFO → TX FIFO → uart_tx로 1클럭씩 전달
7	TX IDLE → START	tx_start = 1	START 진입, tx = 0
8	TX 비트 간격	START 이후	각 비트 16 tick (104.16 µs)
9	TX 시프트	tx_data → data_reg 래치	16 tick마다 우시프트, tx = data_reg[0]
10	최종 비교	receive_uart 수신	expected_data = result_data (pass)

시뮬레이션 결과

S1 RX 검증 결과

결과 요약 **rx 1->0 시 idle -> start 로 state 변화**

기대 결과와 동일 -> 정상 작동





	state
IDEL	00
START	01
DATA	10
STOP	11

순차 회로
-> 다음 clk의 상승 엣지에 변화

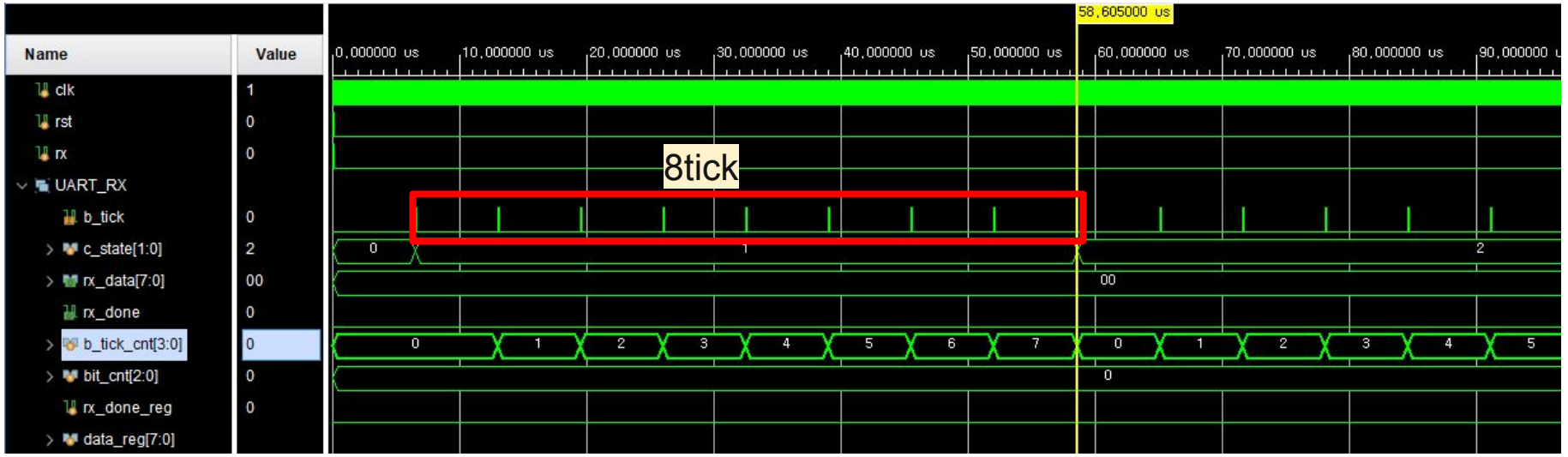
```
for (i = 0; i < 1; i = i + 1) begin
    #10 rst = 0;
    // expected_data = $random() % 256;
    expected_data = 8'b01010101;
    sender_uart(expected_data);
    receive_uart(result_data);
    score_board(result_data, expected_data);
    #100;
end
```

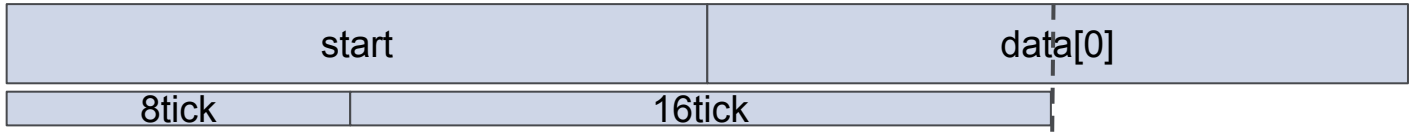
시뮬레이션 결과

S1 RX 검증 결과

결과 요약 **8 tick 후 START -> DATA**

기대 결과와 동일 -> 정상 작동





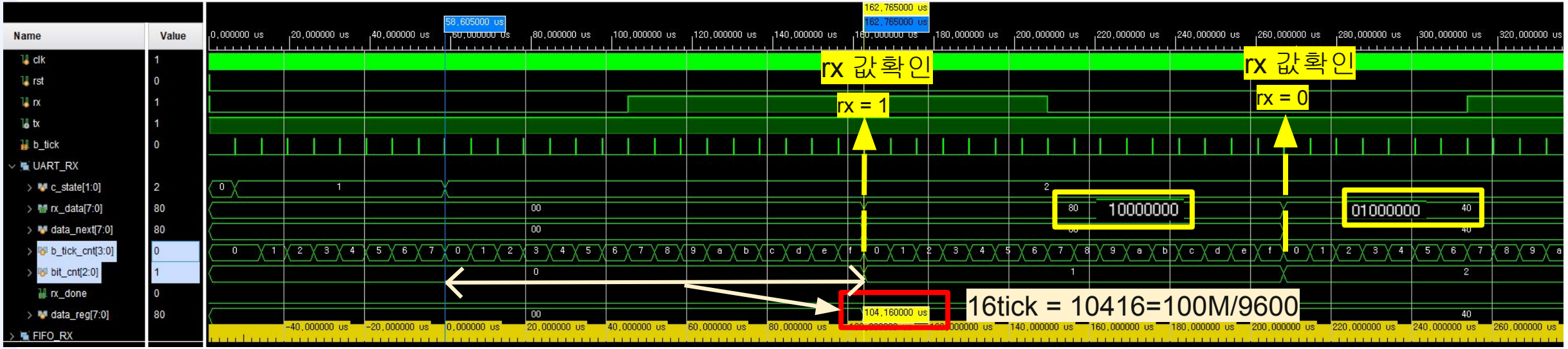
data[0]에서 중간 타이밍에
데이터 읽기 위함

시뮬레이션 결과

S1 RX 검증 결과

결과 요약 **16 tick 간격 | 16tick = buad time | data shift 정상 작동**

기대 결과와 동일 -> 정상 작동

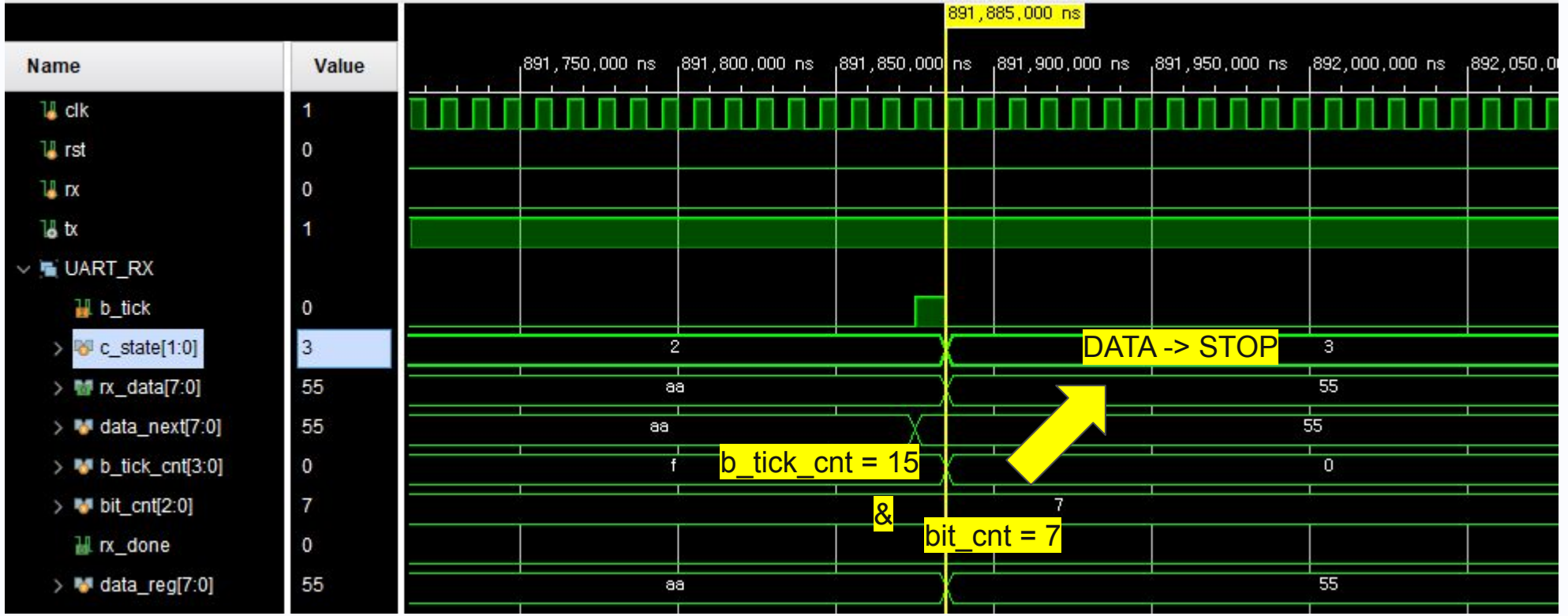


시뮬레이션 결과

S1 RX 검증 결과

결과 요약 **bit_cnt = 7 & b_tick_cnt = 15 시 STOP 진입**

기대 결과와 동일 -> 정상 작동

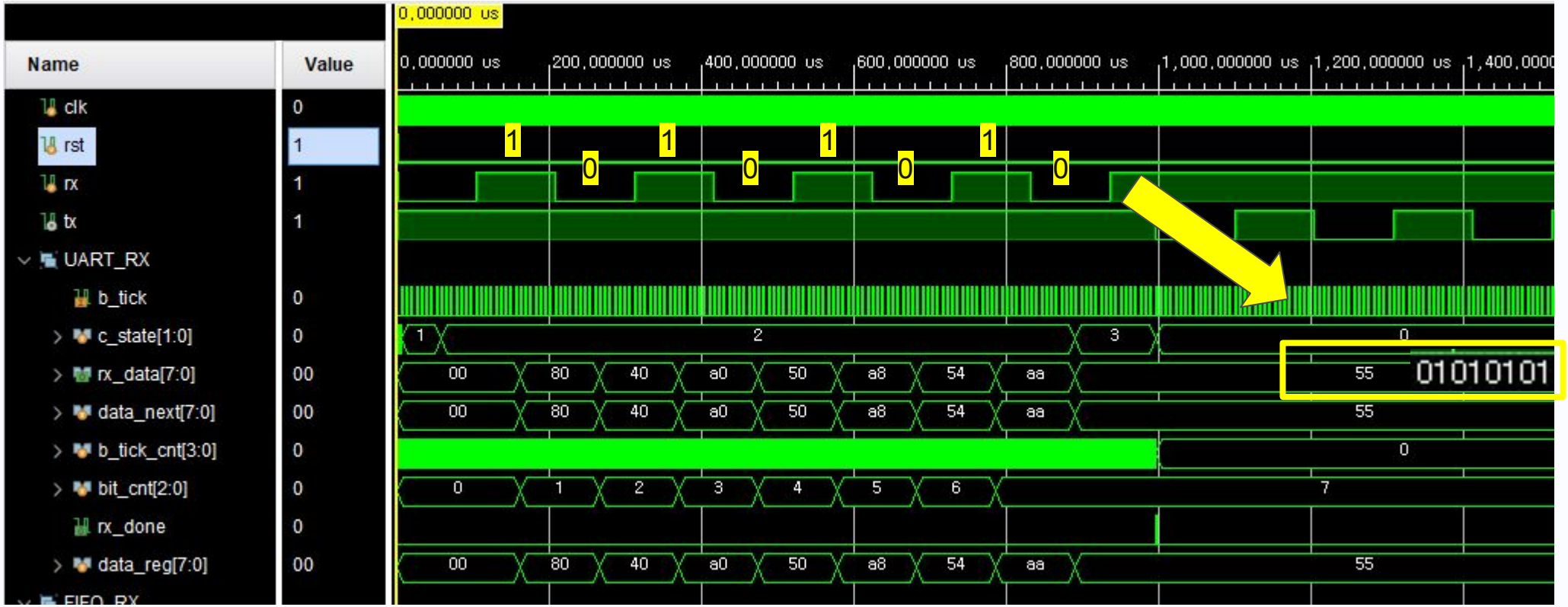


시뮬레이션 결과

S1 RX 검증 결과

결과 요약 입력 : 0101_0101 수신 : 0101_0101

기대 결과와 동일 -> 정상 작동

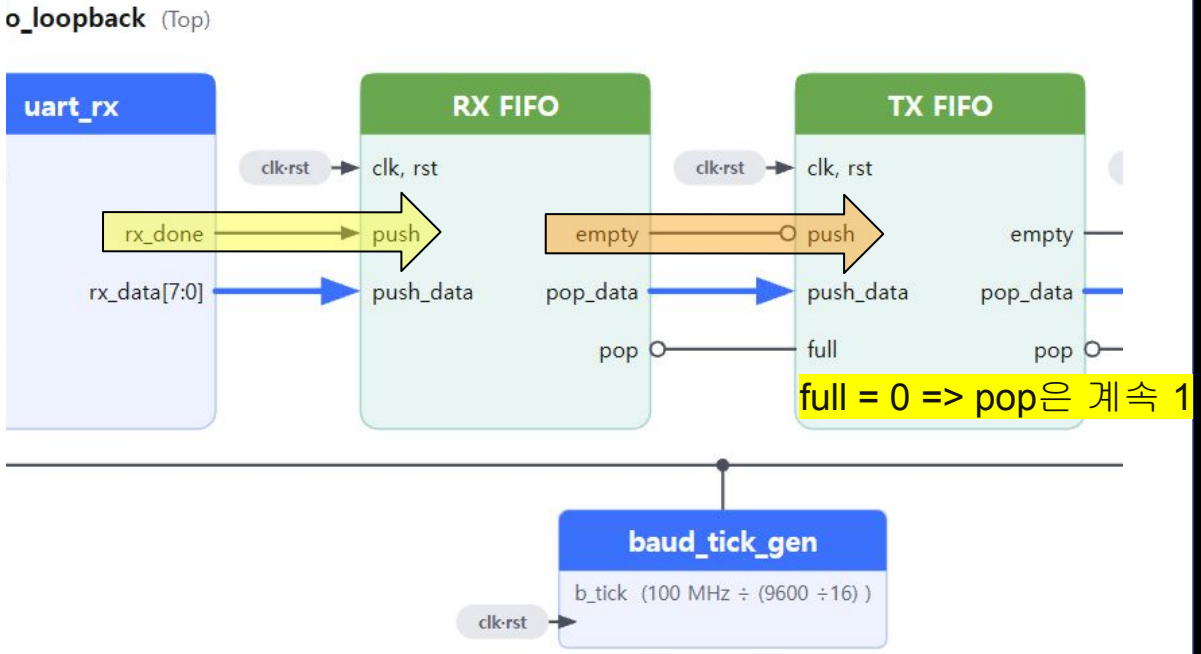


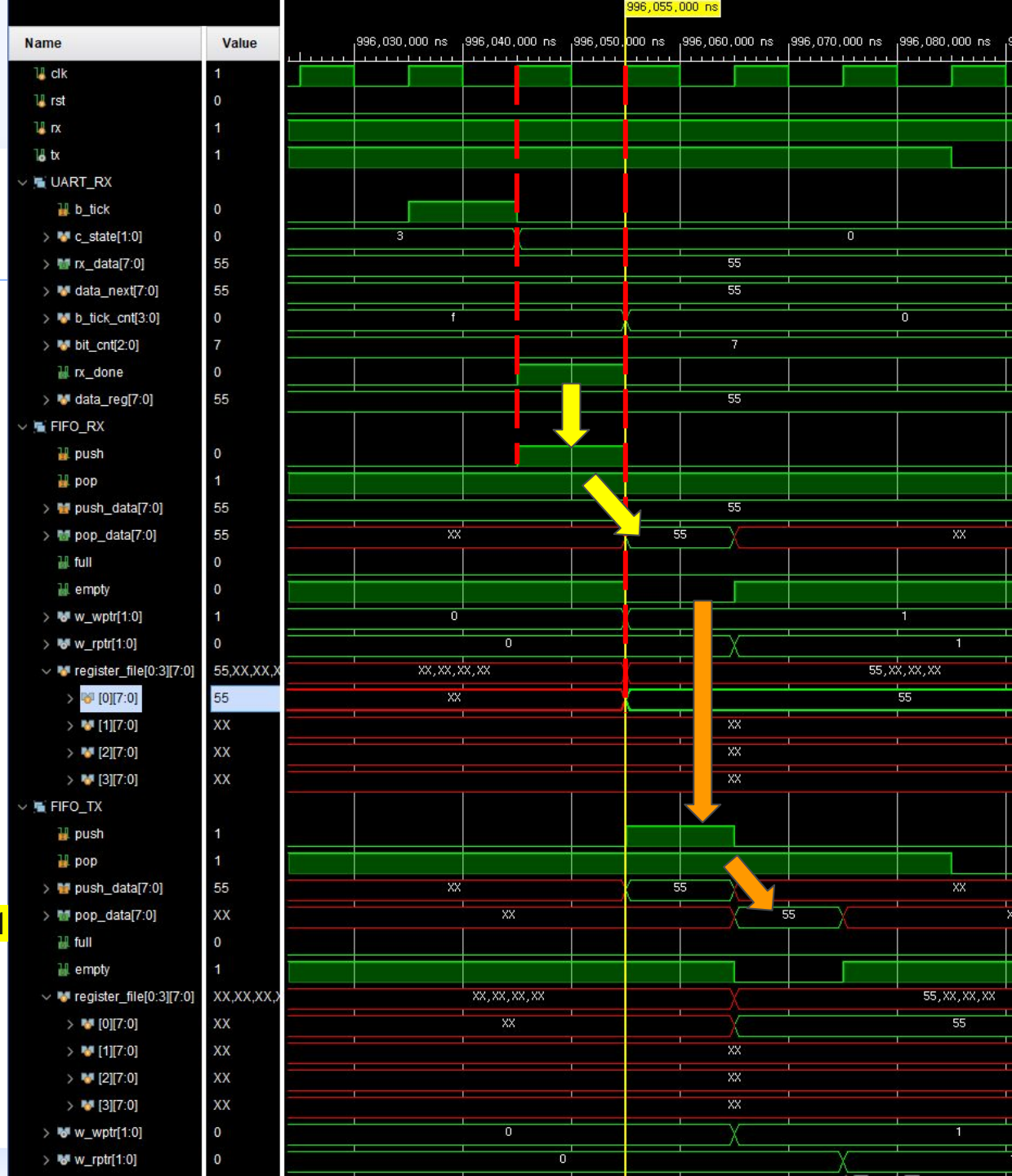
시뮬레이션 결과

S1 fifo 검증 결과

결과 요약 데이터 rx -> rx fifo -> tx fifo

기대 결과와 동일 -> 정상 작동



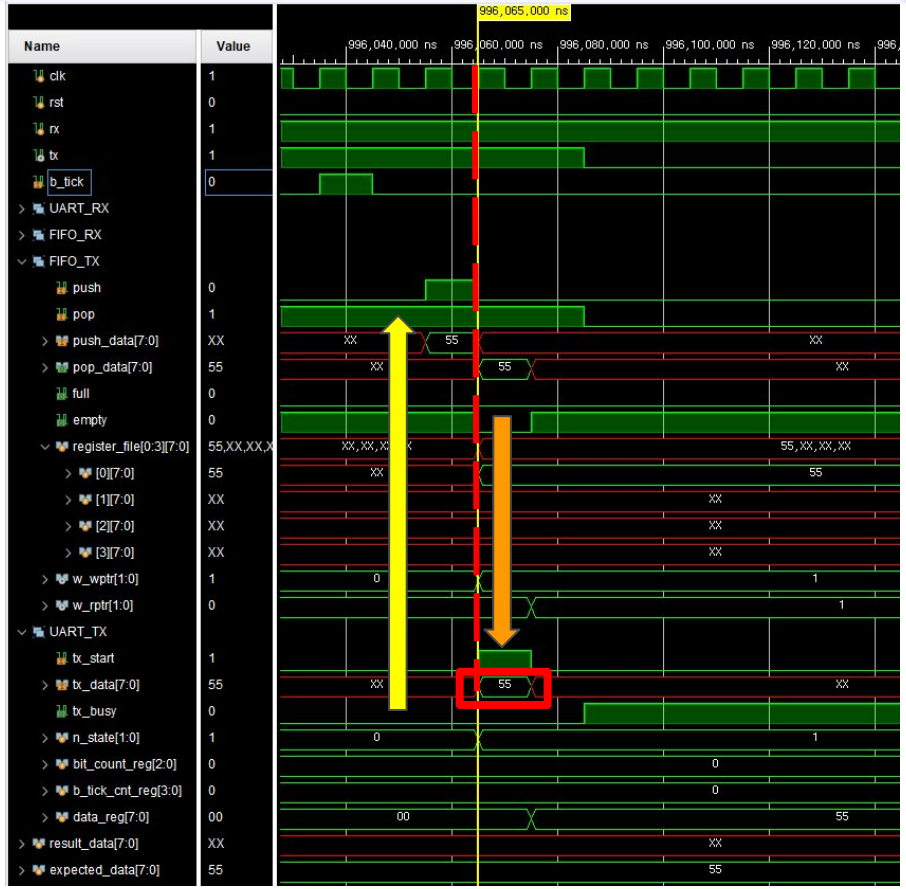


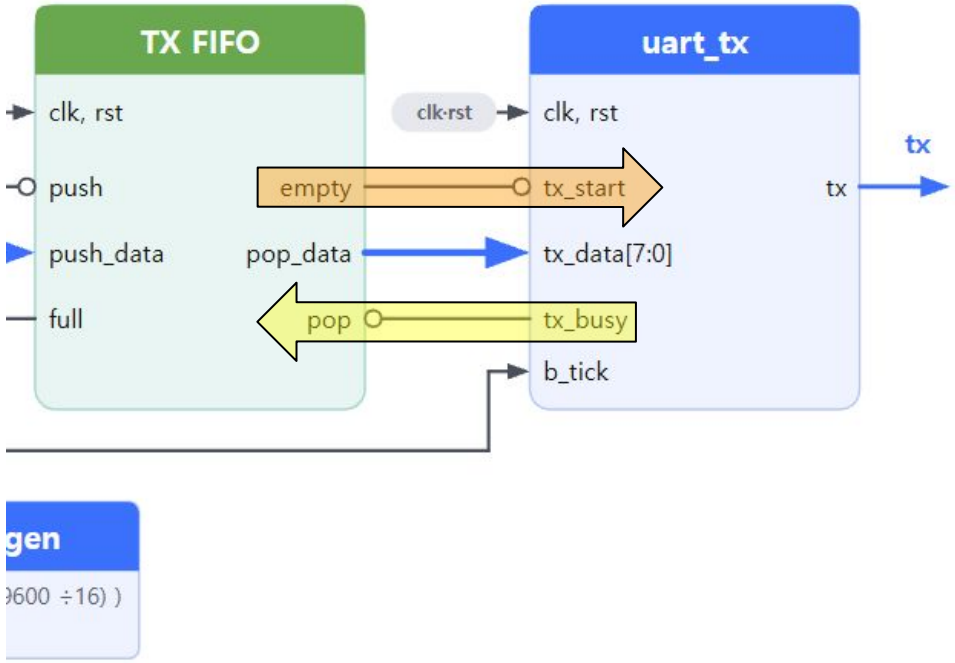
시뮬레이션 결과

S1 fifo검증 결과

결과 요약 데이터 tx fifo -> uart_tx

기대 결과와 동일 -> 정상 작동



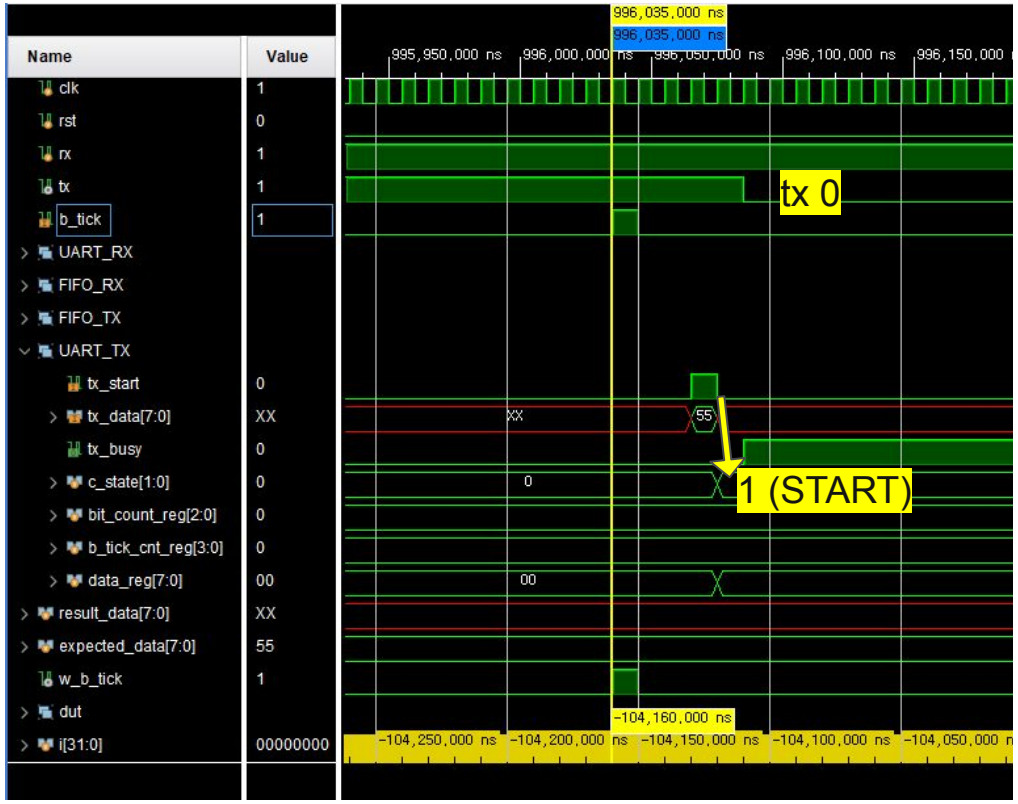


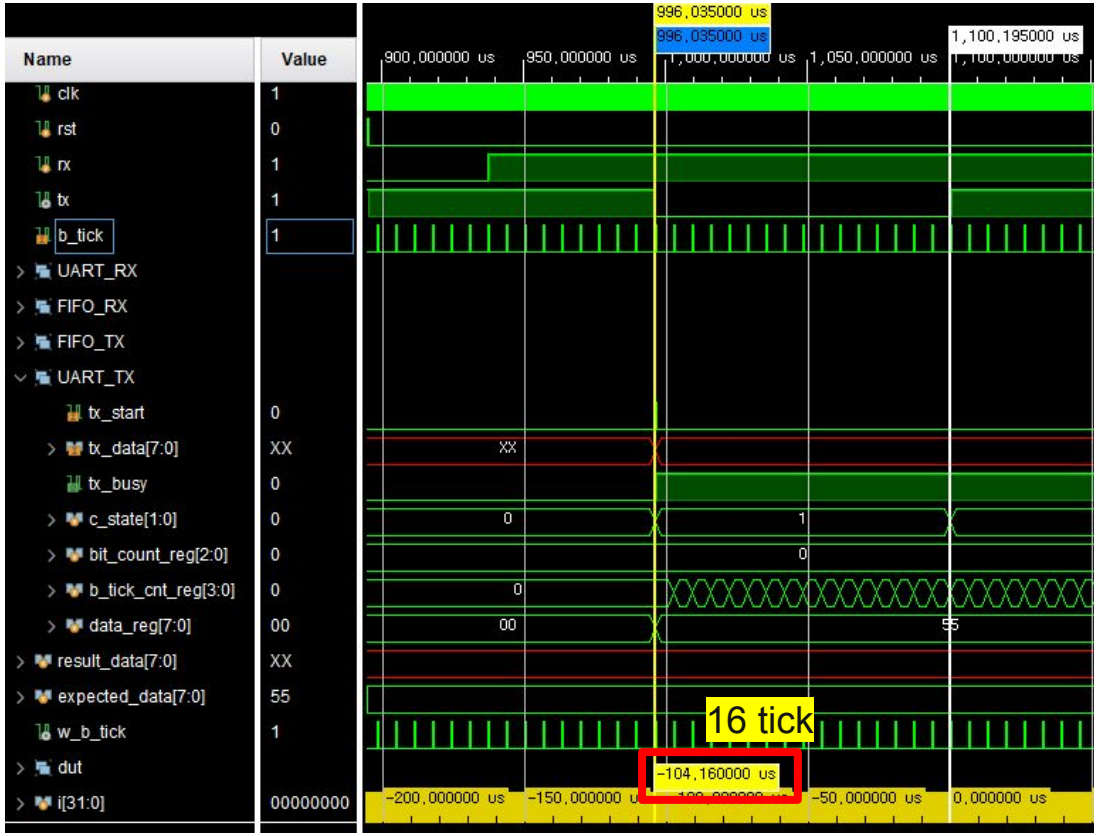
시뮬레이션 결과

S1 TX 검증 결과

기대 결과와 동일 -> 정상 작동

결과 요약 **tx_start =1 => idel -> start , tx = 0 | START 에서 16tick이후->DATA**



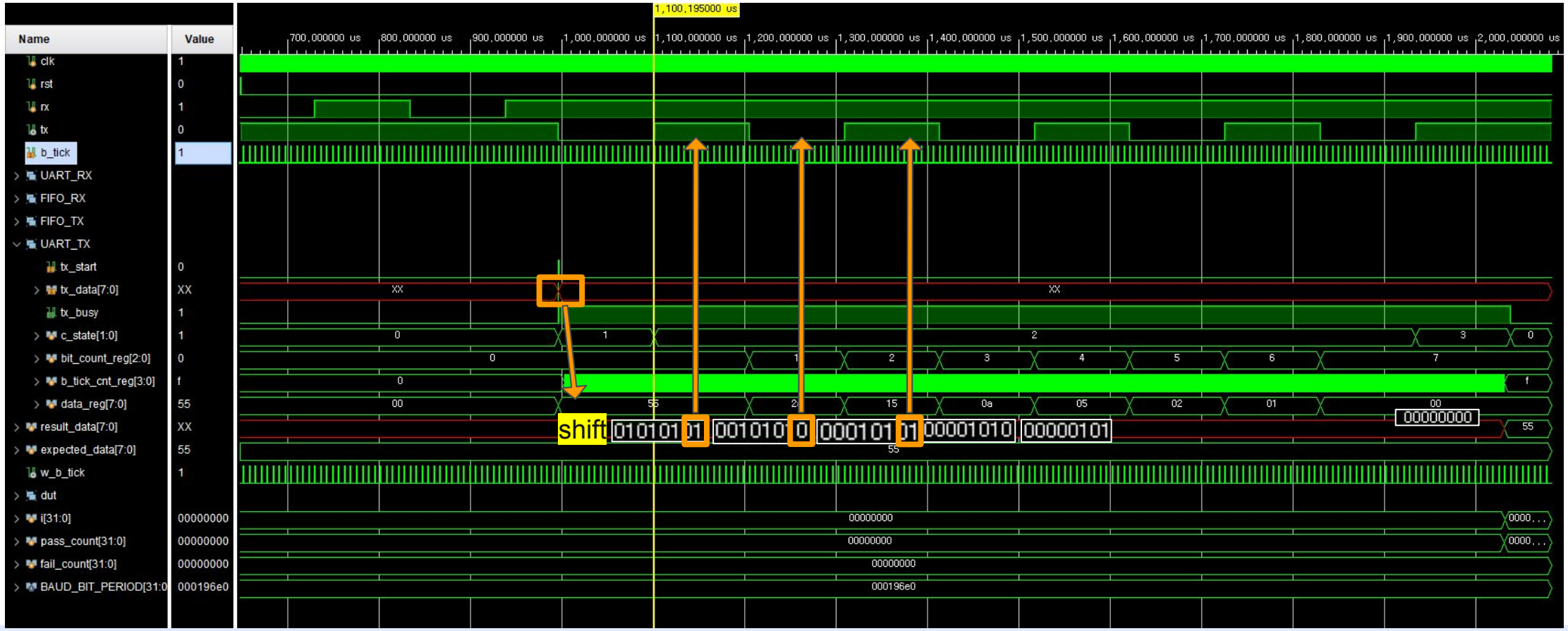


시뮬레이션 결과

S1 TX 검증 결과

결과 요약 **16tick마다 우쉬프트 , tx = data_reg[0]**

기대 결과와 동일 -> 정상 작동

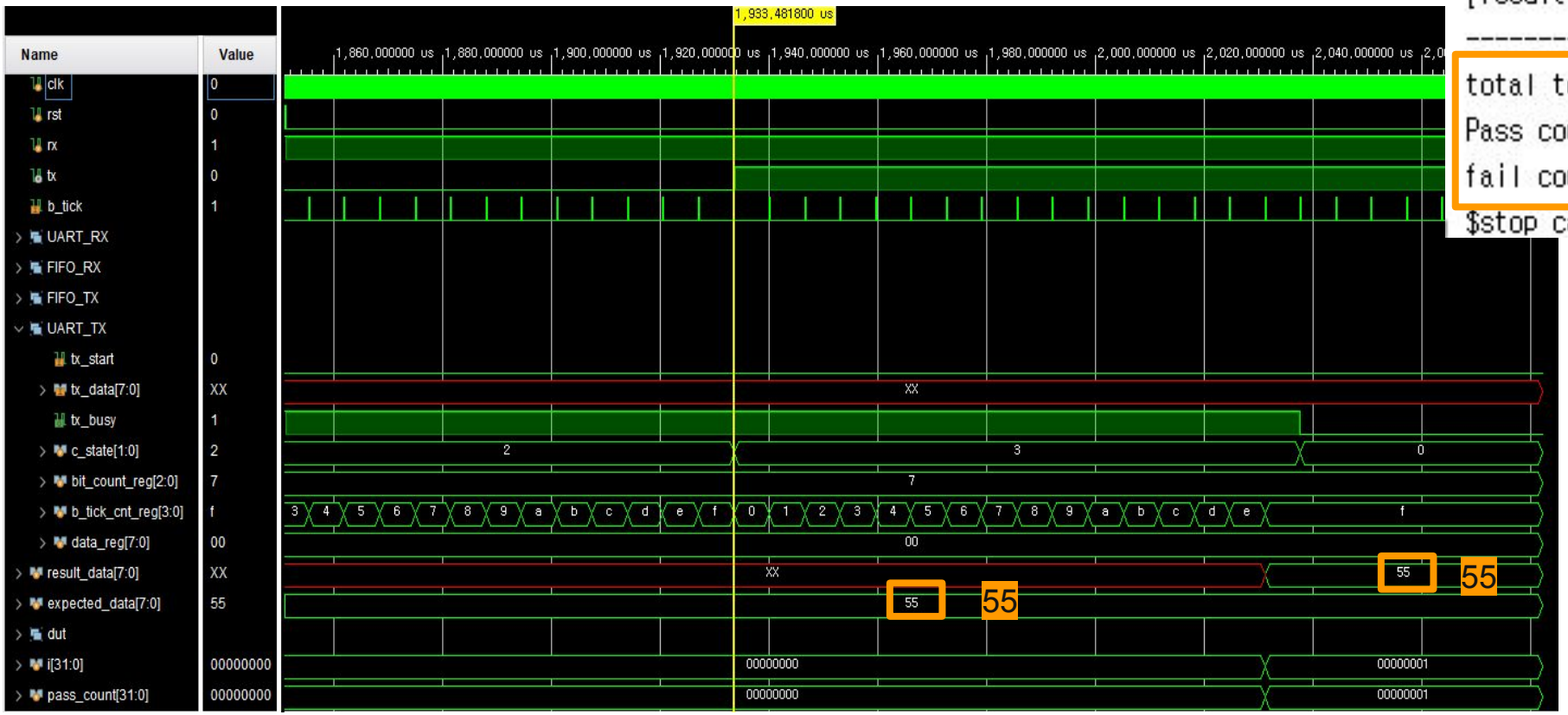


시뮬레이션 결과

S1 TX 검증 결과

결과 요약 **expected_data == result_data | pass**

→ 루프백 구조 정상 작동




```
[result]            2031130000: pass
-------------report------------
total test num =            1
Pass count =                1
fail count =                0
$stop called at time : 2032230 ns : File "D
```

시뮬레이션 시나리오

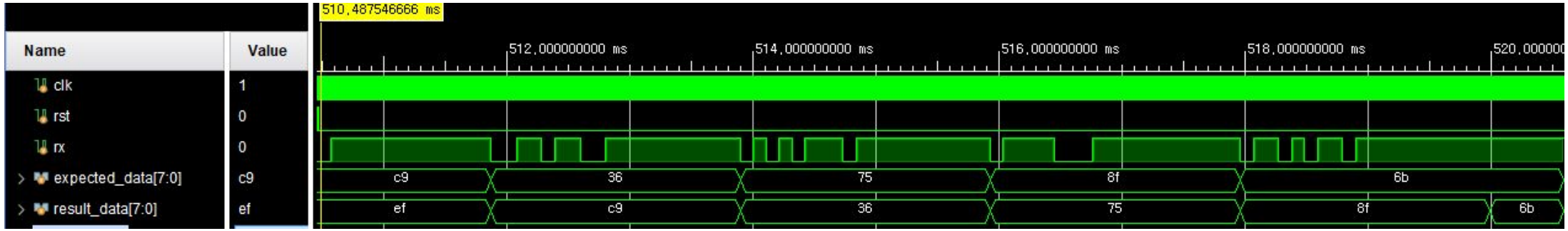
S2 랜덤값 256번 검증

ID	입력 · 상태	입력 · 조건	기대 결과
1	랜덤 데이터 루프백	**$random % 256,** 1바이트씩 송수신	매 회 expected_data = result_data
2	프레임 오류	256회 반복	start · stop bit 오류 메시지 없음
3	결과 집계	256회 종료 후	total 256, pass 256, fail 0

시뮬레이션 결과

S2 랜덤 검증 결과

결과 요약 **256개의 랜덤값 모두 pass** → 루프백 구조 정상 작동




```
[result]        5179b355UUUU: pass
[result]        519994780000: pass
-------------report------------
total test num =         256
Pass count =          256
fail count =            0
$stop called at time : 519995880 ns : F
```

시뮬레이션 시나리오

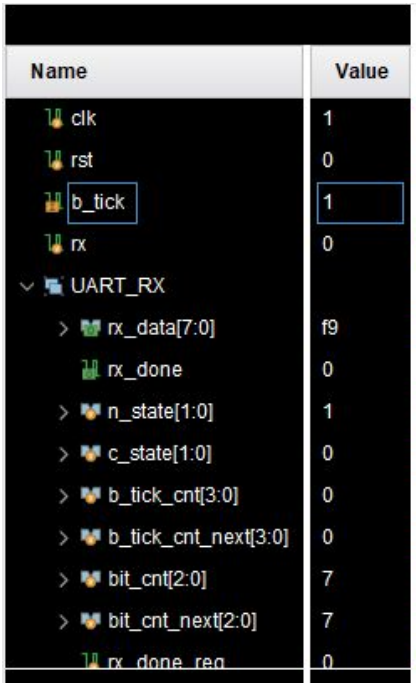
S3 보레이트 오차 허용 범위 검증

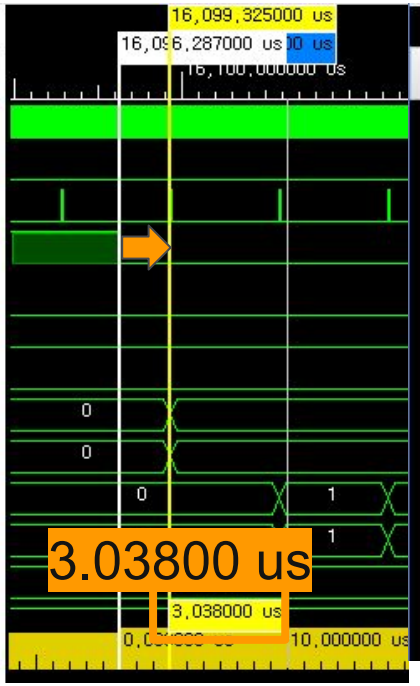
ID	검증 항목	입력 · 조건	기대 결과
1	+ 한계	감지 지연 랜덤 x 오차 : +6.8% ~ +7.4%	+7% 초과 시 일부 fail
2	+ 한계	감지 지연 랜덤 o 오차 : +6.0% ~ +6.5%	+6.2% 초과 시 일부 fail
3	− 한계	감지 지연 랜덤 x 오차 : −5.2% ~ −4.7%	−4.8% 미만 시 일부 fail
4	− 한계	감지 지연 랜덤 o 오차 : −5.1% ~ −4.6%	−4.8% 미만 시 일부 fail

기대 결과의 시간 · 개수는 코드 기준 계산값으로, 파형과 score_board 결과로 확인

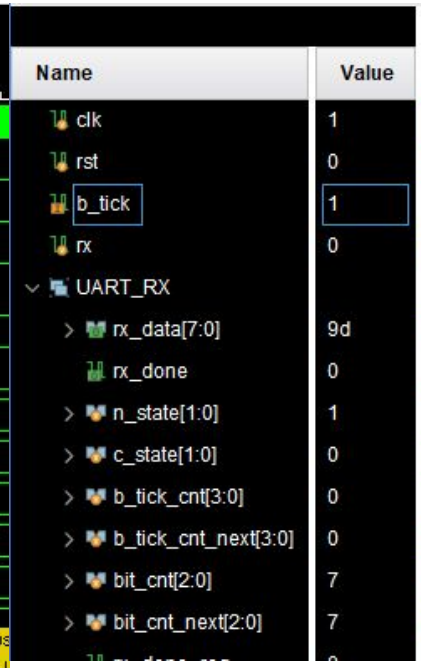
시뮬레이션 결과

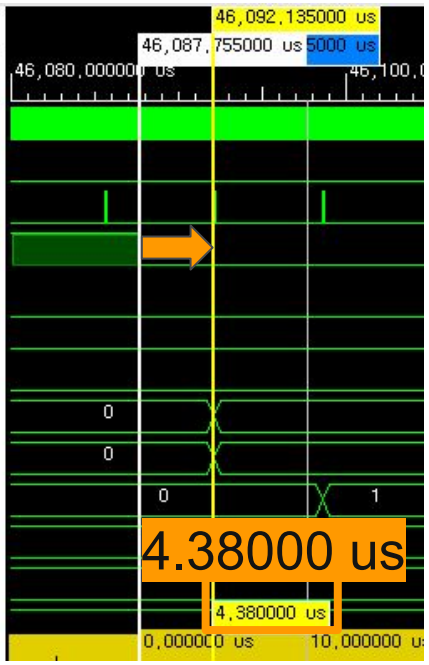
감지 지연



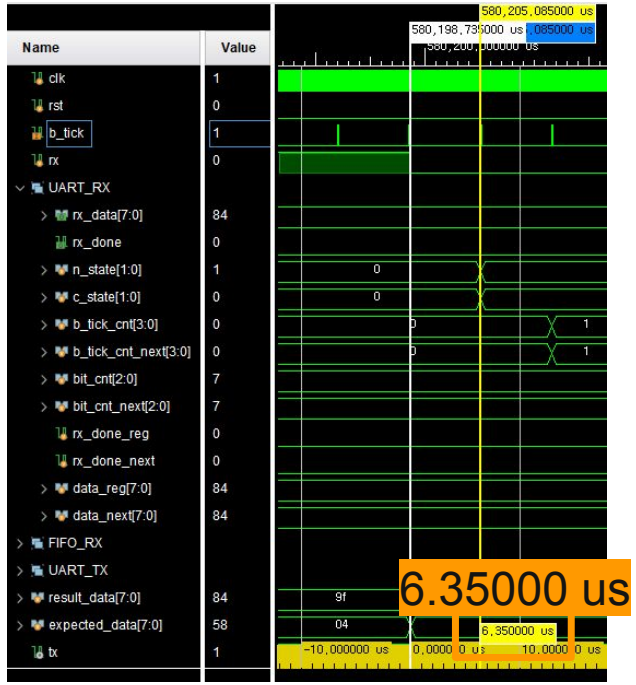


감지지연 랜덤





감지지연 랜덤



감지지연 랜덤 x

-> 다음 바이트의 송신 시작 시점이 **이전 바이트의 tx(= DUT의 tick)에 묶여있음**

시뮬레이션 결과

S3 오차 검증 결과 - 감지 지연 랜덤 O

결과 요약 **0~+6.2% 범위의 오차에서 정상 수신**

```
task sender_uart(input [7:0] send_data);
    integer i;
    begin
        rx = 0;
        #(bit_period);
        for (i = 0; i < 8; i = i + 1) begin
            rx = send_data[i];
            #(bit_period);
        end
        rx = 1;  //stop bit
        #(bit_period);
    end
endtask
```

```
for (j = 60; j < 69; j = j + 1) begin
    pass_count = 0;
    fail_count = 0;
    bit_period = BAUD_BIT_PERIOD * (1.0 + j / 1000.0);
    rst = 1;
    rx = 1;
    #100 rst = 0;
    #(BAUD_BIT_PERIOD * 2);  // 안정화 대기
    for (i = 0; i < 256; i = i + 1) begin
        #10 rst = 0;
        expected_data = $random() % 256;
        // $display("j: %d",j);
        // expected_data = 8'b01010101;
        #({$random} % 6510);
        fork
            begin
                sender_uart(expected_data);
            end
            begin
                receive_uart(result_data);
            end
        join
        score_board(result_data, expected_data);
        #100;
    end
```

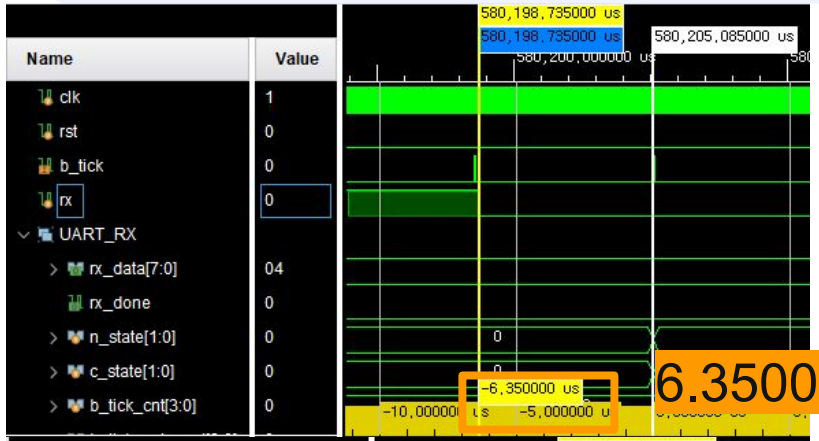
랜덤 감지 지연

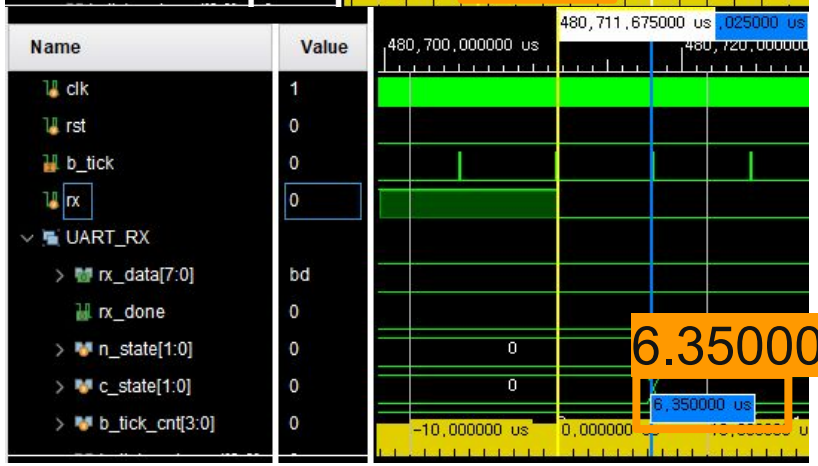
```
-------------report -------------
skew = 6.000000%, bit_period = 110409.6 ns
total test num =         256
Pass count =         256
fail count =           0
-------------report -------------
skew = 6.100000%, bit_period = 110513.8 ns
total test num =         256
Pass count =         256
fail count =           0
-------------report -------------
skew = 6.200000%, bit_period = 110617.9 ns
total test num =         256
Pass count =         256
fail count =           0
-------------report -------------
skew = 6.300000%, bit_period = 110722.1 ns
total test num =         256
Pass count =          51
fail count =         205
-------------report -------------
skew = 6.400000%, bit_period = 110826.2 ns
total test num =         256
Pass count =          14
fail count =         242
-------------report -------------
skew = 6.500000%, bit_period = 110930.4 ns
total test num =         256
Pass count =           4
fail count =         252
```

시뮬레이션 결과

S3 오차 검증 결과 - 감지 지연 랜덤 X

결과 요약 **0~+6.9% 범위의 오차에서 정상 수신**





감지지연 동일

-> 다음 바이트의 송신 시작 시점이 **이전 바이트의 tx(= DUT의 tick)에 묶여 있기** 때문

```
-------------report -------------
skew = 6.800000%, bit_period = 111242.9 ns
total test num =         256
Pass count =         256
fail count =           0
-------------report -------------
skew = 6.900000%, bit_period = 111347.0 ns
total test num =         256
Pass count =         256
fail count =           0
-------------report -------------
skew = 7.000000%, bit_period = 111451.2 ns
total test num =         256
Pass count =           1
fail count =         255
-------------report -------------
skew = 7.100000%, bit_period = 111555.4 ns
total test num =         256
Pass count =           5
fail count =         251
-------------report -------------
skew = 7.200000%, bit_period = 111659.5 ns
total test num =         256
Pass count =           2
fail count =         254
$stop called at time : 1898888375 ns : File "D:/w
```

시뮬레이션 결과

S3 오차 검증 결과 - 감지 지연 랜덤 O

결과 요약 **-4.8%~0 범위의 오차에서 정상 수신**



감지지연 랜덤

```
-------------report ------------
skew = -5.200000%, bit_period = 98743.7 ns
total test num =         256
Pass count =         197
fail count =          59
-------------report ------------
skew = -5.100000%, bit_period = 98847.8 ns
total test num =         256
Pass count =         206
fail count =          50
-------------report ------------
skew = -5.000000%, bit_period = 98952.0 ns
total test num =         256
Pass count =         227
fail count =          29
-------------report ------------
skew = -4.900000%, bit_period = 99056.2 ns
total test num =         256
Pass count =         251
fail count =           5
-------------report ------------
skew = -4.800000%, bit_period = 99160.3 ns
total test num =         256
Pass count =         256
fail count =           0
-------------report ------------
skew = -4.700000%, bit_period = 99264.5 ns
total test num =         256
Pass count =         256
fail count =           0
$stop called at time : 3051244445 ns : File "D:/wo
```

시뮬레이션 결과

S3 오차 검증 결과 - 감지 지연 랜덤 X

결과 요약 **-4.8%~0 범위의 오차에서 정상 수신**

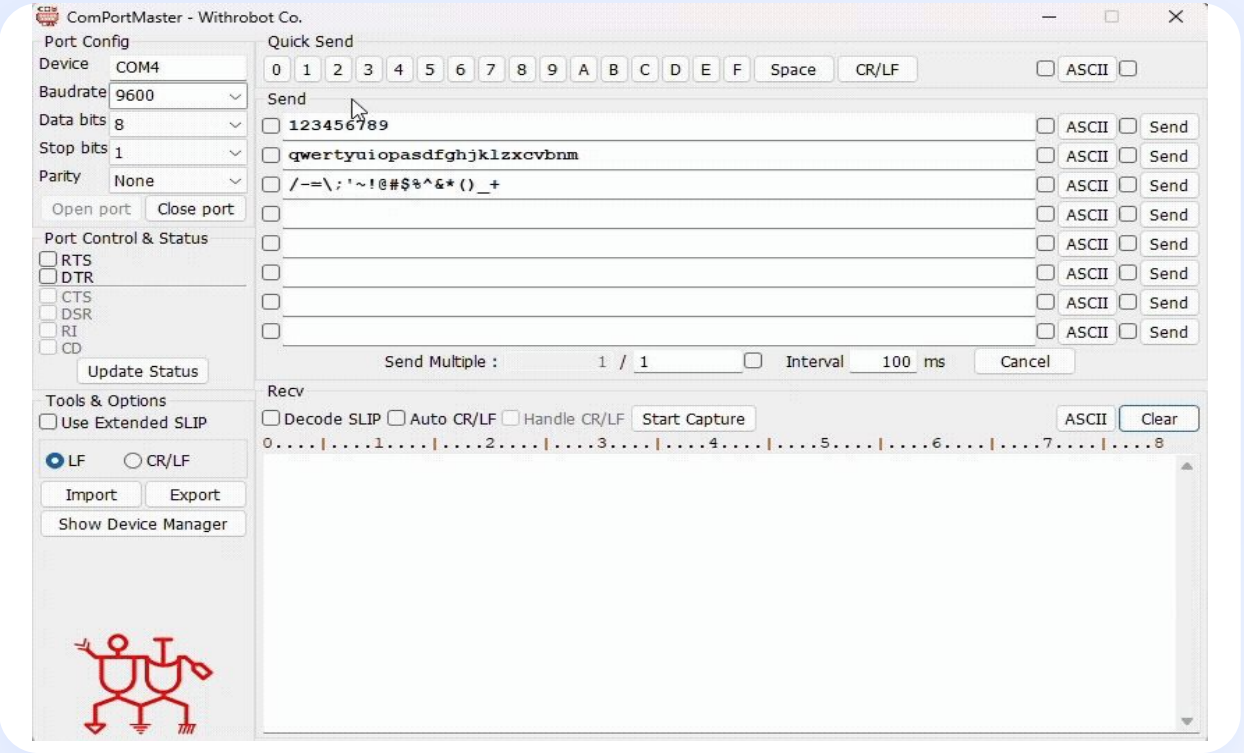
랜덤 0의 오차와 동일



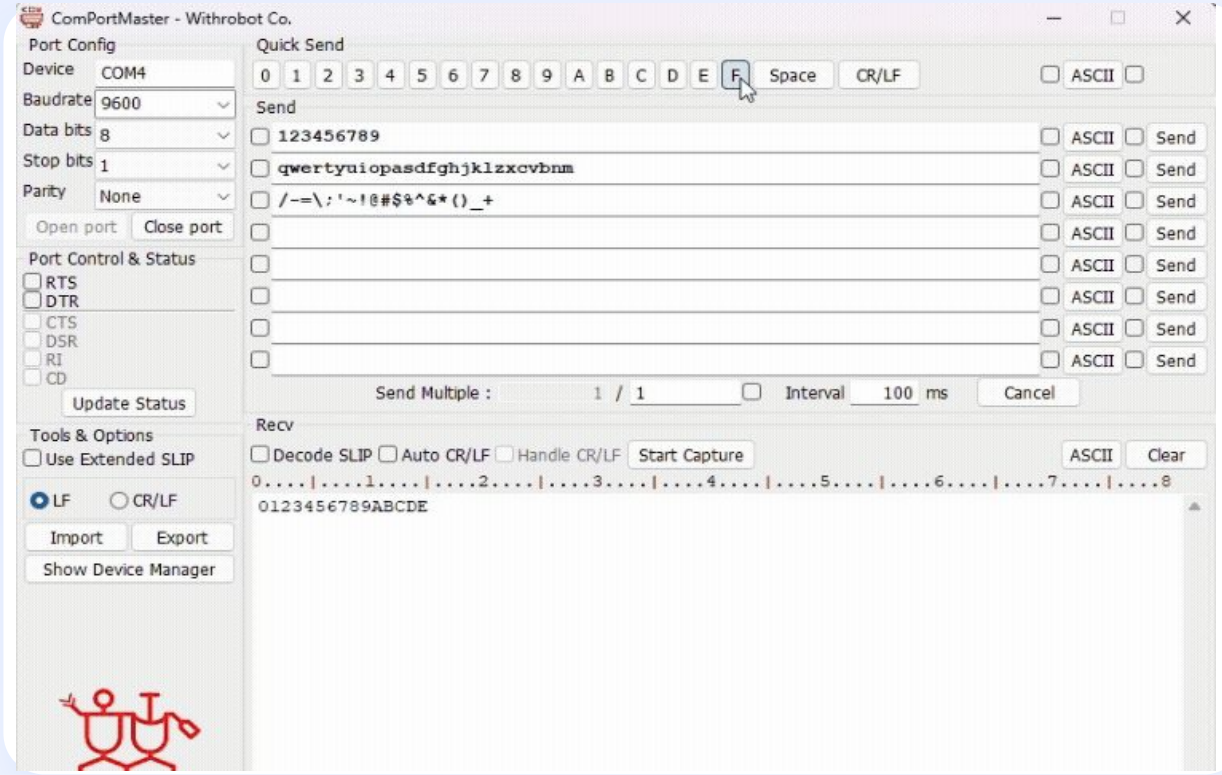
감지지연 동일 & 감지 지연 최악 상태
=> 감지 지연 랜덤과 오차 동일

```
-------------report ------------
skew = -5.100000%, bit_period = 98847.8 ns
total test num =         256
Pass count =         129
fail count =         127
-------------report ------------
skew = -5.000000%, bit_period = 98952.0 ns
total test num =         256
Pass count =         121
fail count =         135
-------------report ------------
skew = -4.900000%, bit_period = 99056.2 ns
total test num =         256
Pass count =         141
fail count =         115
-------------report ------------
skew = -4.800000%, bit_period = 99160.3 ns
total test num =         256
Pass count =         256
fail count =           0
-------------report ------------
skew = -4.700000%, bit_period = 99264.5 ns
total test num =         256
Pass count =         256
fail count =           0
-------------report ------------
skew = -4.600000%, bit_period = 99368.6 ns
total test num =         256
Pass count =         256
fail count =           0
$stop called at time : 3559572525 ns : File "D:
```

동작 영상



단일 바이트 전송
=>정상 작동



연속 바이트 전송
=>정상 작동

결론

결론 구현 성과

01 UART RX · TX + FIFO 루프백 시스템 구현

- 100 MHz Basys 3에서 **9600 bps 8N1**로 PC가 보낸 데이터를 RX FIFO → TX FIFO를 거쳐 그대로 반환
- ComportMaster로 실제 보드 동작 확인

02 16배 오버샘플링으로 비트 중앙 샘플링

- baud_tick_gen : 100 MHz ÷ **651** → b_tick 주기 **6.51 µs** (1 bit = 16 tick)
- start bit 감지 후 **8 tick**, 이후 **16 tick마다** 샘플링해 비트 중앙 부근에서 데이터 수신

03 시뮬레이션으로 기능과 마진 검증

- S1 단일 바이트(0x55) : RX → FIFO → TX 전 경로의 상태 전이 · 타이밍 확인 | S2 랜덤 256회 루프백 : **전부 pass**
- S3 송신 보레이트 오차 **−4.8% ~ +6.2%** 에서 감지 위상과 무관하게 100% 정상 수신 → USB-UART(±2% 이내) 대비 충분한 마진

고찰

고찰 배운점

01 감지 지연이 샘플링 위치를 결정한다

- IDLE에서 b_tick마다만 rx를 확인 → start bit 감지에 **0 ~ 1 tick** 지연 발생
- 샘플링은 감지 시점부터 세므로 실제 위치는 비트 중앙이 아닌 **50 ~ 56%** → 16배 오버샘플링은 이 오차를 1/16 bit 이내로 줄이는 방법

02 오차 허용 범위는 비대칭이다

- 샘플링 위치가 중앙보다 약간 뒤 → 느린 송신(**+6.2%**)을 빠른 송신(**−4.8%**)보다 더 잘 견딤
- 첫 fail(+6.3%, −4.9%)이 D7 샘플링 위치로 계산한 이론 한계(**+6.25%, −4.86%**)와 일치

03 테스트벤치 구조가 검증 결과를 바꾼다

- 1 bit = 16 tick 정수배라 송신 시작이 항상 tick 직후에 고정 → 모든 바이트가 d ≈ 1 tick으로만 시험되어 + 한계가 **약 7%로 과대평가**
- 0 ~ 1 tick **랜덤 지연**으로 위상을 분산해 최악 조건 한계 확인 → tb가 실제 환경을 얼마나 재현하는지가 중요

THANK YOU

감사합니다

질의응답 · Q&A

